



PDF Download
3704253.3706138.pdf
07 March 2026
Total Citations: 0
Total Downloads: 200



Published: 10 January 2025

Citation in BibTeX format

PEPM '25: 2025 ACM SIGPLAN
International Workshop on Partial
Evaluation and Program Manipulation
January 21, 2025
CO, Denver, USA

Conference Sponsors:
SIGPLAN

DL Latest updates: <https://dl.acm.org/doi/10.1145/3704253.3706138>

RESEARCH-ARTICLE

Typed Program Analysis without Encodings

BARRY JAY

Typed Program Analysis without Encodings

Barry Jay

Unaffiliated

Australia

Barry.jay8@gmail.com

Abstract

Programs are viewed as both functions to be executed and data structures to be analysed, but this has always required encoding, e.g. of a lambda-term to a syntax tree, so that a self-interpreter could not be applied to itself directly, but only to its code. Further, the code of a typed program should have a distinctive type. In a tree calculus, however, program analysis can be supported directly, without encodings, including the self-application of a breadth-first self-interpreter of type

$$\mathbf{bf} : \forall X. \forall Y. (X \rightarrow Y) \rightarrow (X \rightarrow Y) .$$

CCS Concepts: • Theory of computation $\rightarrow$ Lambda calculus.

Keywords: typed self-interpreter, combinators, tree calculus

ACM Reference Format:

Barry Jay. 2025. Typed Program Analysis without Encodings. In *Proceedings of the 2025 ACM SIGPLAN International Workshop on Partial Evaluation and Program Manipulation (PEPM '25), January 21, 2025, Denver, CO, USA*. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3704253.3706138>

1 Introduction

Programs are viewed as both functions to be executed and data structures to be analysed. For example, an *interpreter* [25] may be given by a program **int** that is a function of programs viewed as data structures, including **int** itself [2, 4–6, 8, 22, 26, 30, 32, 33, 35, 36, 38]. Actually, interpreters are canonical examples of program analyses that: exercise the whole language; stand in for program compilation; and try to fold the meta-theory of computing into its theory, e.g. [24].

Unfortunately, traditional models of computation support only one of these two viewpoints. For example, Turing’s machines are programs-as-functions, which must be encoded as tapes [41]. Conversely, if the tape is primary, as in a *partial*

combinatory algebra [12], then decoding is required to expose the machinery. Again, a program p in a *lambda-calculus* [1, 9] or *combinatory logic* [10, 15, 37] is a function whose structure is revealed by encoding it as a syntax tree $[p]$. In that setting, self-interpretation is given by **int** [int]. Since the encodings use meta-theory that is not definable within the calculi, they are incomplete for program analysis. Thus, program analysis is usually static: source code is parsed into syntax trees, which are analysed before being converted to, say lambda abstractions and machine code. Dynamic program analysis is then equivalent to analysing the tape of a Turing machine, which is not compositional. A more subtle approach is to use *staged compilation*, e.g. [29, 34] but still there is an irreversible flow from data structures to functions.

Even when the syntax tree is available, it has been hard to reason about, or even to give it a type without resorting to extensions of System F_ω or higher kinds [5, 6, 8, 35]. If p is a program of type $U \rightarrow V$ then its encoding $[p]$ will require its own *expression type* $\mathbf{Exp}\{U \rightarrow V\}$ which adds an extra layer of complexity. Second, there are two or three natural choices of type for the interpreter [22]. The type $\mathbf{Exp}\{U \rightarrow V\} \rightarrow U \rightarrow V$ is that of a *self-recogniser* that decodes an expression back to its original program. The type

$$\mathbf{Exp}\{U \rightarrow V\} \rightarrow \mathbf{Exp}\{U\} \rightarrow \mathbf{Exp}\{V\}$$

is that of a *self enactor* which produces a function of expressions. The latter is more interesting, as it allows one to impose an evaluation strategy on reduction. Third, since encodings can be nested, one is obliged to consider expression types of expression types in an unbounded hierarchy. All of these difficulties are driven by the use of encodings.

These difficulties have seemed unavoidable. Not only are the traditional calculi *extensional*, unable to distinguish different implementations of the same function, but the recursive functions have been represented by terms that do *not* have normal forms. However, it turns out that combinatory logic can indeed represent recursive functions by normal forms [17], which may then be identified with the programs. That is, the notions of program and computation, which have been identified in lambda-calculus, can now be separated. All of the μ -recursive arithmetic functions [27] required for Turing completeness can be represented by programs in combinatory logic. In this sense, combinatory logic is complete for numerical analysis. However, it is not complete for program analysis, being unable to analyse its own programs, its own

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. PEPM '25, January 21, 2025, Denver, CO, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1350-7/25/01

<https://doi.org/10.1145/3704253.3706138>

normal forms, even to separate different representations of the identity function.

Program analysis requires the *intensional computation* [18] of, say, a *factorisation calculus* (such as *SF-calculus*) [21] or that of [22] or a *tree calculus* [19]. Among these, tree calculus is more natural, in the following sense. The computations in *SF-calculus* are represented by syntax trees with nodes labelled by *S* or *F*. Since *S* and *F* both act when given three arguments, i.e. are ternary operators, the programs are given by binary trees labelled by *S* or *F*. However, the choice of labels is artificial, not natural like the numbers, which makes for poor semantics. By contrast, a tree calculus has only one operator Δ called node, so that labels are redundant, and the syntax trees become *natural trees*, i.e. without labels. Since Δ is a ternary operator, the programs become the binary natural trees. The original tree calculus has three reduction rules, for the application of Δ to a leaf, stem or fork. Presumably the results in this paper can be developed there, but it is easier to work with a variant called *triage calculus* suggested by Johannes Bader in conversation. Its five reduction rules can be summarised by the mnemonic

$$\begin{aligned} \Delta\Delta &= K \\ \Delta(\Delta x) &= Sx \\ \Delta(\Delta wx)y &= \mathbf{triage}\{w, x, y\} . \end{aligned}$$

Extensional functions are supported by the use of *K* and *S*, while *triage* supports intensionality: the $\mathbf{triage}\{w, x, y\}$ specialises to *w*, *x* or *y* according to whether its next argument *z* is a leaf, stem or fork. This can be used to decide the size of a program, or equality of programs. Arguably, tree calculus is complete for program analysis. Certainly, it supports self-interpreters such as the *breadth-first* interpreter **bf** which can be applied to itself directly, as **bf bf**.

This paper reprises previous work on tree calculus (which has yet to survive peer review) and then goes on to develop a type theory for it. A high point of this is a type

$$\mathbf{bf} : \forall X. \forall Y. (X \rightarrow Y) \rightarrow (X \rightarrow Y)$$

for a self-interpreter.

Any type system for tree calculus must allow a single program to take both a data type and a function type. The principal type of each program is a *tree type* such as the type $\mathbf{F}(\mathbf{S}(\mathbf{S} \mathbf{L}))(\mathbf{S} \mathbf{L})$ built from type forms for leaves (**L**) and stems (**S**) and forks (**F**). Any function type for a program is then given by *subtyping* of the tree type. For example, consider the reduction

$$Ku v \rightarrow u .$$

Here $K = \Delta\Delta$ so that $Ku = \Delta\Delta u : \mathbf{F} \mathbf{L} U$ if $u : U$. To apply Ku to $v : V$ requires that $\mathbf{F} \mathbf{L} U$ be a subtype of some type $V \rightarrow T$. Further, for the reduction rule above to preserve typing, *T* must be *U*. So we have the subtyping axiom

$$\mathbf{F} \mathbf{L} U < V \rightarrow U .$$

In this manner, each of the reduction rules of the calculus inspires a subtyping axiom.

Once subtyping is involved, it can be made to do all of the interesting work, in that two type derivation axioms will suffice to type all programs, with one rule for applications and one that can type a leaf by any subtype of **L** (see Figure 2). Thus, the expressive power of the type system is determined by that of subtyping, which will grow throughout this paper. The general flavour of the approach is exemplified by the treatment of recursive functions.

In lambda calculus, the *Y* combinator is defined to be $\omega\omega$ where

$$\omega = \lambda x. \lambda f. f(xxf) .$$

This supports the reduction $Yf \rightarrow f(\omega\omega f) = f(Yf)$ expected of a fixpoint combinator. However, this combinator *Y* does not have a normal form, as it reduces to $\lambda f. f(Yf)$ which contains itself as a sub-expression. Thus *Y* can reduce to ever larger terms. It follows that programs in lambda calculus are unstable, and that program analysis is forced to work with encodings of programs as syntax trees.

According to Church's Thesis, combinatory logic is equivalent to lambda calculus, so one might expect the same problems to arise there, as when *Y* is defined to be the self-application of the translation $S(K(SI))(SII)$ of ω . However, combinatory logic supports finer control over evaluation than lambda calculus. By a process akin to η -expansion of f to $\lambda x. fx$, one can replace an application MN by $\mathbf{wait}\{M, N\}$ given by

$$S(S(KM)(KN))I$$

since $\mathbf{wait}\{M, N\}x$ reduces to MNx . Now $\mathbf{wait}\{M, N\}$ is a normal form if *M* and *N* are, so that every combinator is functionally equivalent to a normal form! In particular, we can define the combinator *Y* to be the normal form

$$Y = \mathbf{wait}\{\omega, \omega\} .$$

Unfortunately, this does not mean that Yf is a normal form when *f* is, since $Yf \rightarrow f(Yf)$ as before.

Rather, given a binary function *f* we can construct a *fix-point function* $Z\{f\}$ that is normal if *f* is, yet satisfies

$$Z\{f\} x \rightarrow f Z\{f\} x .$$

It is defined to be of the form $\mathbf{wait}2\{\omega_2, \omega_2, f\}$ where $\mathbf{wait}2$ is analogous to $\mathbf{wait}$ and ω_2 is a normal form analogous to ω . In order to show $f : (U \rightarrow V) \rightarrow (U \rightarrow V)$ implies $Z\{f\} : U \rightarrow V$ it is enough to show that $\omega_2\omega_2 : \mathbf{Fix}(U \rightarrow V)$ where $\mathbf{Fix} T = (T \rightarrow T) \rightarrow T$. Supposing that there is a canonical type Ω_2 of ω_2 , it is enough to add the subtyping axiom

$$\Omega_2 < \Omega_2 \rightarrow \mathbf{Fix}(U \rightarrow V) .$$

Thus, even in traditional combinatory logic, all of the μ -recursive arithmetic functions necessary for Turing completeness can be expressed as normal forms, and be simply-subtyped using type constants **Bool** and **Nat** and Ω_2 for booleans, natural numbers and the type of ω_2 .

This subtyping rule is not inspired by the deduction rules of some logic, or the reduction rules of the calculus. Rather, it captures the correctness of the implementation of Z . That is, the proof that reduction preserves typing will recapitulate the proof that the $Z\{f\}$ meets its specification. As the sophistication of the correctness proofs increases, so too will that of the subtyping rules. For example, generic queries will require their own subtyping rule, to type examples such as

$$\begin{aligned} \text{size} &: \forall X. X \rightarrow \text{Nat} \\ \text{equal} &: \forall X. \forall Y. X \rightarrow Y \rightarrow \text{Bool} . \end{aligned}$$

In turn, this will not be enough to type a self-evaluator **bf** that requires its first argument to be of function type. Instead we will show it has type $\forall Z. Z \rightarrow \mathbf{A} Z$ where $\mathbf{A} Z$ records the expectation that Z will become a function type. By specialising Z to $X \rightarrow Y$, the expectation is met and we can prove

$$\mathbf{bf} : \forall X. \forall Y. (X \rightarrow Y) \rightarrow (X \rightarrow Y) .$$

Note that we now have a mixture of subtyping and the parametric polymorphism of universal quantifiers $\forall X$. Such mixtures are rare, but not unheard of [7, 16, 39]. Here, the instantiation of quantified types will be characterised by a subtyping rule.

It follows that there are many type systems for tree calculus, parametrised by the choice of types and subtyping relation. Each extension considered here will be introduced in its own section of the paper, with theorems asserting that the examples are well typed. Note however, that although the theorems are true for the type system then in play, the formal verification in **Coq** [3] (soon to be known as **Rocq**) will be given only once, for the richest type system, as defined in Section 9. For ease of concordance, the theorem names in the paper are the same as in the **Coq** verification.

The structure of the paper is as follows. Section 1 is the introduction. Section 2 shows how to type traditional combinatory logic so that all μ -recursive functions are given by typed normal forms. Section 3 introduces the variant of tree calculus used in this paper and shows some basic properties of its reduction system that will be used later to show correctness of **bf**. Sections 4 – 8 introduce examples of programs, prove that they meet their specifications, and give an informal account of their typing. In particular: Section 4 introduces lambda-abstractions; Section 5 introduces the booleans, the natural numbers and tagging; Section 6 introduces fixpoint functions; Section 7 introduces generic queries, exemplified by **size** and **equal**; and Section 8 introduces a self-interpreter. Section 9 introduces the richest type system we will consider, able to handle all of the earlier examples. Section 10 proves that the type system is sound, in that reduction preserves typing. Section 11 draws conclusions and considers future work.

2 Combinatory Logic

This section will show how traditional combinatory logic is able to represent all of the μ -recursive arithmetic functions, i.e. the functions required for Turing completeness, as normal forms, and give them types in a simply-typed system, equipped with subtyping.

The *combinations* of *SK-calculus*, also here called *traditional combinatory logic*, are

$$M, N ::= S \mid K \mid MN .$$

The *factorable forms* are the combinations of the form S, SM, SMN, K and KM . The *programs* are the factorable forms whose sub-expressions are also programs. Its two reduction rules are

$$\begin{aligned} SMNP &\longrightarrow MP(NP) \\ KMN &\longrightarrow M . \end{aligned}$$

The reduction relation is the congruence generated by the reduction rules. That is, it is closed under application and is reflexive and transitive.

Some examples of programs and program constructions are

$$\begin{aligned} I &= SKK \\ \text{wait}\{M, N\} &= S(S(KM)(KN))I \\ \text{wait2}\{M, N, P\} &= S(S(S(KM)(KN))(KP))I \\ \omega_2 &= S(K(S(SKK))) \\ &\quad (S(S(KS)(S(K(S(KS)))) \\ &\quad (S(S(KS)(S(KK)(S(KS) \\ &\quad (S(S(KS)K)K))))(KK))) \\ &\quad (K(K(SKK))) \\ Z\{f\} &= \text{wait2}\{\omega_2, \omega_2, f\} . \end{aligned}$$

For example, we have

$$\begin{aligned} \text{wait}\{M, N\}x &= S(S(KM)(KN))Ix \\ &\longrightarrow S(KM)(KN)x(Ix) \\ &\longrightarrow (KMx)(KNx)x \\ &\longrightarrow MNx . \end{aligned}$$

Thus, $\text{wait}\{M, N\}$ is a factorable form that has the same functional behaviour as MN and so is a program if M and N are. Similarly, we have $\text{wait2}\{M, N, P\}x \longrightarrow MNPx$ for any terms M, N, P and x .

The combination ω_2 can be thought of as a representation of the lambda-abstraction

$$\lambda w. \lambda f. f \text{ wait2}\{w, w, f\} .$$

Thus, $Z\{f\}$ is a program if f is, and

$$Z\{f\}x \longrightarrow \omega_2 \omega_2 f x \longrightarrow f(\omega_2 \omega_2 f)x = f Z\{f\} x$$

shows that $Z\{f\}$ is a fixpoint function for f .

The representation of booleans and natural numbers is traditional, with

$$\begin{aligned} \mathbf{tt} &= K \\ \mathbf{ff} &= KI \\ \mathbf{zero} &= KI \\ \mathbf{succ} &= S(S(KS)K) \end{aligned}$$

where the successor **succ** may be thought of as representing $\lambda n.\lambda f.\lambda x.f(nfx)$. Then pairing, conditionals, composition, primitive recursion and minimisation can all be defined in the usual manner, on the understanding that minimisation uses Z to ensure that all the μ -recursive functions are represented by programs. Now let us see how to type them.

The *simple types* are here given by

$$T, U, V ::= \mathbf{Bool} \mid \mathbf{Nat} \mid \Omega_2 \mid U \rightarrow T .$$

That is they are function types built from the type constants **Bool** (for booleans), **Nat** (for natural numbers) and Ω_2 (for ω_2). The intended semantics of the type constants is captured by the *subtyping relation*, whose rules are given by

$$\begin{aligned} \mathbf{Bool} &< U \rightarrow U \rightarrow U \\ \mathbf{Nat} &< (U \rightarrow U) \rightarrow (U \rightarrow U) \\ \Omega_2 &< \Omega_2 \rightarrow \mathbf{Fix}\{U \rightarrow V\} \end{aligned}$$

where $\mathbf{Fix}\{T\} = (T \rightarrow T) \rightarrow T$. The subtyping of **Bool** supports conditionals. The subtyping of **Nat** supports iterators. The subtyping of Ω_2 supports fixpoint functions. The subtyping relation is the reflexive, transitive relation generated by these rules and closed under the implication

$$\frac{U_2 < U_1 \quad V_1 < V_2}{U_1 \rightarrow V_1 < U_2 \rightarrow V_2} .$$

Type derivation is given by some axioms for typing **tt** etc plus a rule for typing applications. Here are some special cases of the axioms for any types U, V, W and T :

$$\begin{aligned} S &: (U \rightarrow V \rightarrow W) \rightarrow (U \rightarrow V) \rightarrow (U \rightarrow W) \\ K &: U \rightarrow V \rightarrow U \\ \mathbf{tt} &: \mathbf{Bool} \\ \mathbf{ff} &: \mathbf{Bool} \\ \mathbf{zero} &: \mathbf{Nat} \\ \mathbf{succ} &: \mathbf{Nat} \rightarrow \mathbf{Nat} \\ \omega_2 &: \Omega_2 . \end{aligned}$$

The general versions of the axioms may replace the type by any supertype of it. For example, the general rule for **tt** is

$$\mathbf{tt} : T \quad \text{if } \mathbf{Bool} < T .$$

The rule for typing applications is

$$\frac{M : U \rightarrow V \quad N : U}{MN : V} .$$

$$\begin{aligned} \Delta\Delta yz &\longrightarrow y \\ \Delta(\Delta x)yz &\longrightarrow xz(yz) \\ \Delta(\Delta wx)y\Delta &\longrightarrow w \\ \Delta(\Delta wx)y(\Delta u) &\longrightarrow xu \\ \Delta(\Delta wx)y(\Delta u v) &\longrightarrow yuv . \end{aligned}$$

Figure 1. Reduction Rules of Triage Calculus

It follows that if $M : U$ is any type derivation and $U < V$ then $M : V$.

Theorem 2.1 (SK_reduction_preserves_typing). *For all combinations M and N and type T , if $M : T$ and $M \longrightarrow N$ then $N : T$.*

3 Tree Calculus

The *combinations* of a tree calculus are given by

$$M, N ::= \Delta \mid MN .$$

The operator Δ (pronounced "node") is a ternary operator. When unapplied, Δ is a *leaf* while combinations of the form ΔM are *stems* and of the form ΔMN are *forks*. Together these constitute the *factorable forms*. The *programs* are the factorable forms whose sub-expressions (branches) are also programs.

There is some flexibility in the choice of reduction rules. The original tree calculus [19] uses just three rules but they are hard to reason about, or to type. This paper introduces a variant tree calculus called *trriage calculus* that better separates concerns by using the five reduction rules in Figure 1.

In shorthand, they become

$$\begin{aligned} \Delta\Delta &= K \\ \Delta(\Delta x) &= Sx \\ \Delta(\Delta wx)y &= \mathbf{trriage}\{w, x, y\} \end{aligned}$$

on the understanding that K and S behave as in combinatory logic, while *trriage* is the novelty, that performs case analysis on its argument, with cases for leaves, stems and forks. Indeed, these equations can be reversed to define K and *trriage* but some care is required with S (defined below) since Sx merely reduces to $\Delta(\Delta x)$.

Some examples of program constructions are

$$\begin{aligned} K &= \Delta\Delta \\ S_1\{p\} &= \Delta(\Delta p) \\ S &= S_1\{K\Delta\}\Delta \\ I &= S_1\{K\}K \\ \mathbf{trriage}\{w, x, y\} &= \Delta(\Delta wx)y \\ \mathbf{swap}\{f\} &= S_1\{K(S_1\{f\})\}K \\ \mathbf{wait}\{M, N\} &= S_1\{S_1\{KM\}(KN)\}I \\ \mathbf{wait2}\{M, N, P\} &= S_1\{S_1\{S_1\{KM\}(KN)\}(KP)\}I . \end{aligned}$$

$$\frac{\text{Leaf} < T}{\Delta : T} \quad \frac{M : U \rightarrow V \quad N : U}{MN : V}.$$

Figure 2. Type Derivation for Combinators

Note that, unlike $S_1\{p\}$ the combination $S p$ is *not* a program since it is the application of fork. Rather $S p$ reduces to $S_1\{p\}$.

Since there is only one operator, the programs and computations can be thought of as *natural trees*, i.e. trees without any labels on the nodes. Each program p can be represented as a numeral $\text{num}\{p\}$ in ternary arithmetic, using Polish notation, where a node is represented by the number of its arguments. For example, we have

$$\begin{aligned} \text{num}\{\Delta\} &= 0 \\ \text{num}\{K\} &= 10 \\ \text{num}\{S_1\{K\}\} &= 1110 \\ \text{num}\{S\} &= 212000 \\ \text{num}\{I\} &= 211010 \end{aligned}$$

As well as suggesting approaches to implementation based on ternary arithmetic [44], this notation serves as a reminder that all programs, no matter their surface syntax, are binary trees.

This section ends with a couple of conventional theorems. Confluence [40] shows that terms related by reduction have a common reduct; this yields a good equational theory.

Theorem 3.1 (confluence_tree_calculus). *Reduction of triage calculus is confluent.*

Head reduction will be used to show the correctness of the breadth-first evaluator in Section 8. The *head reduction relation* $t \rightarrow_h u$ is weaker than the general reduction relation. Although it also supports the reduction rules, and is reflexive and transitive, reduction of applications is restricted to the following three possibilities: if $t_1 \rightarrow_h t_2$ then

$$\begin{aligned} t_1 u &\rightarrow_h t_2 u \\ \Delta t_1 u v &\rightarrow_h \Delta t_2 u v \\ \Delta (\Delta w u) v t_1 &\rightarrow_h \Delta (\Delta w u) v t_2. \end{aligned}$$

The first possibility allows reduction at the head of an application: the other two are required to support the intensional reduction rules that must inspect their arguments.

Theorem 3.2 (head_reduction_to_factorable_form). *Reduction to factorable form can always begin with head reduction. That is, for all terms t and v , if $t \rightarrow_h v$ and v is factorable then there is a term u such that $t \rightarrow_h u$ and $u \rightarrow v$.*

Type derivation for the combinations is given by the two rules in Figure 2. These rules are parametrised by the choice

$$\begin{aligned} L &< U \rightarrow S U \\ S U &< V \rightarrow F U V \\ F L U &< V \rightarrow U \\ F (S (U \rightarrow V \rightarrow T)) (U \rightarrow V) &< U \rightarrow T \\ F (F T V) W &< L \rightarrow T \\ F (F U (V \rightarrow T)) W &< S V \rightarrow T \\ F (F U V) (W_1 \rightarrow W_2 \rightarrow T) &< F W_1 W_2 \rightarrow T \end{aligned}$$

Figure 3. Minimal Subtyping Axioms

of types and their subtyping relation, which determine expressive power. A minimal collection of types for tree calculus is given by

$$T, U, V ::= L \mid S U \mid F U V \mid U \rightarrow V.$$

Here L is the type of a leaf and $S U$ is the type of a stem whose branch is of type U and $F U V$ is the type of a fork whose branches have types U and V respectively and $U \rightarrow V$ is the type of functions from U to V .

A minimal collection of subtyping axioms is given in Figure 3. The subtyping relation is generated from the axioms by closing under the type formation rules (function types are contravariant in their arguments) and closing under reflexivity and transitivity.

For example, if $M : U$ and $N : V$ then $\Delta M : S U$ and $\Delta MN : F U V$ by applying the first two axioms. Similarly, $K : S L$ implies $K M : F L U < V \rightarrow U$ by the third axiom. Again, I of type $F (S (S L)) (S L)$ is also of type $U \rightarrow U$ for any type U . Also, for any types U, V and T the combinator S can be shown to have type $(U \rightarrow V \rightarrow T) \rightarrow (U \rightarrow V) \rightarrow U \rightarrow T$.

It may seem strange to limit subtyping to leaf types in Figure 2 but this makes proofs easier without compromising expressivity, since the general subsumption rule

$$\frac{M : U \quad U < V}{M : V}$$

follows by induction on the structure of M , as in Theorem 9.1 in Section 9.

Theorem 3.3 (derive_basic). *For every type U, V and T we have*

$$\begin{aligned} K &: U \rightarrow V \rightarrow U \\ S &: (U \rightarrow V \rightarrow T) \rightarrow (U \rightarrow V) \rightarrow (U \rightarrow T) \\ I &: T \rightarrow T \end{aligned}$$

and

$$\frac{f : U \rightarrow V \rightarrow T}{\text{swap}\{f\} : V \rightarrow U \rightarrow T}$$

and

$$\frac{M : U \rightarrow V \rightarrow T \quad N : U}{\text{wait}\{M, N\} : V \rightarrow T}.$$

and

$$\frac{M : U \rightarrow V \rightarrow W \rightarrow T \quad N : U \quad P : V}{\text{wait2}\{M, N, P\} : W \rightarrow T}.$$

Each program p has a canonical type $\text{progtty}(p)$ defined by

$$\begin{aligned} \text{progtty}\{\Delta\} &= L \\ \text{progtty}\{\Delta q\} &= S \text{progtty}\{q\} \\ \text{progtty}\{\Delta qr\} &= F \text{progtty}\{q\} \text{progtty}\{r\}. \end{aligned}$$

Theorem 3.4 (`programs_have_types`). $p : \text{progtty}(p)$ for each program p .

4 Lambda-Abstraction

In practice, it is convenient to support lambda abstraction $\lambda x.t$ of a term t by a term variable x . This is achieved by adding term variables to the combinators to get the *terms*

$$t, u ::= x \mid \Delta \mid t u$$

where x is from a collection of term variables $x, y, z \dots$ that support decidable equality. Since variables are not bound in the BNF, it is trivial to decide if a variable is free in a term or not, and to define the *substitution* $\{u/x\}t$ of a term u for a variable x in a term t . Then we can *define* lambda-abstraction by

$$\begin{aligned} \lambda x.x &= I \\ \lambda x.y &= K y \quad (y \neq x) \\ \lambda x.\Delta &= K \Delta \\ \lambda x.tu &= K(tu) \quad (\text{if } x \text{ is not free in } tu) \\ \lambda x.tu &= \Delta(\Delta(\lambda x.t))(\lambda x.u) \quad (\text{if } x \text{ is free in } tu). \end{aligned}$$

Note that the fourth line above introduces some light optimisation to the definition. Note that the right-hand side of the last line is a reduct of $S(\lambda x.t)(\lambda x.u)$.

Theorem 4.1 (`star_beta`). For all terms t and u and term variable x

$$(\lambda x.t)u \longrightarrow \{u/x\}t.$$

Now type derivation requires a context Γ that maps term variables to types. Given such a type context and a term variable x and a type U then $\Gamma, x : U$ is the context that maps x to U and maps any other variable y to $\Gamma(y)$. The corresponding type derivation axioms are given in Figure 4. These rules are enough to type abstractions with respect to the minimal set of subtyping rules as follows.

Theorem 4.2 (`derive_star`). For all type contexts Γ and term variables x and terms t

$$\frac{\Gamma, x : U \vdash t : T}{\Gamma \vdash \lambda x.t : U \rightarrow T}.$$

$$\frac{\Gamma(x) < T}{\Gamma \vdash x : T} \quad \frac{\text{Leaf} < T}{\Gamma \vdash \Delta : T}$$

$$\frac{\Gamma \vdash t : U \rightarrow V \quad \Gamma \vdash u : U}{\Gamma \vdash t u : V}.$$

Figure 4. Type Derivation with Contexts

We will assume the presence of term variables and type contexts from now on, even when they are not strictly necessary.

5 Booleans, Natural Numbers and Tagging

Quantified types, of the form $\forall X.T$ where X is a type variable and T is a type, will be used to type some polymorphic examples. Quantification by some sequence of type variables $\vec{X}$ may be written as $\forall \vec{X}.T$. The free type variables of T , and the substitution $\{U/X\}T$ of a type U for X in T is defined in the usual way, to avoid variable capture. The resulting subtyping rules are

$$\begin{aligned} \forall X.T &< \{U/X\}T \\ T &< \forall X.T \quad (\text{if } X \text{ is not free in } T). \end{aligned}$$

In the **Coq** verification, type variables are represented by *deBruijn indices*. That is, type variables are named by natural numbers, and the quantified type “Quant T ” or $\forall T$ binds the variable with index 0. Then the second subtyping rule above becomes $T < \forall(\text{lift } 1 T)$ where `lift 1` raises all indices by one, to avoid variable capture. However, in this paper we will work with named bindings, as in $\forall X.T$.

Types for booleans and natural numbers could be introduced as type constants, as in Section 2, but it is more convenient to represent them as quantified function types, as in System F [13], by

$$\begin{aligned} \text{Bool} &= \forall X.X \rightarrow X \rightarrow X \\ \text{Nat} &= \forall X.(X \rightarrow X) \rightarrow (X \rightarrow X) \end{aligned}$$

so that we can infer

$$\begin{aligned} \text{true} &= K : \text{Bool} \\ \text{false} &= KI : \text{Bool} \\ \text{zero} &= KI : \text{Nat} \\ \text{succ} &= S_1\{S_1\{K\Delta\}(S_1\{K\Delta\}K)\} : \text{Nat} \rightarrow \text{Nat} \end{aligned}$$

As well as supporting all the earlier theorems of this paper, this is enough to define primitive recursion and all of the other μ -recursive functions [27] except for minimisation, which requires general recursion.

Note that **false** and **zero** are both represented by KI which may lead to confusion. This can be avoided by *tagging* terms with their types, or indeed any other information of interest, without changing the functional behaviour of the term.

Given a term t and another term i which is to be its tag, define

$$\text{tag}\{t, i\} = \text{wait}\{Kt, i\}.$$

It has the same functional behaviour as t since

$$\text{wait}\{Kt, i\}x \longrightarrow Ktix \longrightarrow tx$$

and i can be recovered by applying **untag** given by

$$\text{trriage}\{\Delta, \Delta, \text{trriage}\{\Delta, \text{trriage}\{\Delta, \Delta, \text{trriage}\{\Delta, KI, \Delta\}, \Delta\}\}, \Delta\}.$$

untag can be thought of as pattern matching on the structure of $\text{tag}\{t, i\} = S_1\{S_1\{K^2t\}(Ki)\}I$ where $K^2t = K(Kt)$. It reduces as follows:

$$\text{untag tag}\{t, i\} = KI (K^2t) (Ki) I = (Ki) I = i.$$

In this manner, typed tree calculus could be embedded within untyped tree calculus. A thorough development of this approach is beyond the scope of this paper.

6 Recursive Programs

Recursive functions will be of the form $Z\{f\}$ as defined in Section 2.

Theorem 6.1 (Z_{red}). *For all terms f and x ,*

$$Z\{f\} x \longrightarrow f Z\{f\} x.$$

The typing of $Z\{f\}$ proceeds much as before, except that now Ω_2 is defined to be **progt** $\{\omega_2\}$ and we will generalise from $U \rightarrow V$ to $\forall \vec{X}.U \rightarrow V$ to get the subtyping axiom

$$\Omega_2 < \Omega_2 \rightarrow \text{Fix} (\forall \vec{X}.U \rightarrow V)$$

for any sequence $\vec{X}$ of type variables and types U and V . This implies

Theorem 6.2 (derive_Z). *For all type context Γ and sequence of type variables $\vec{X}$ and types U and V and term f , we can prove*

$$\frac{\Gamma \vdash f : (\forall \vec{X}.U \rightarrow V) \rightarrow (\forall \vec{X}.U \rightarrow V)}{\Gamma \vdash Z\{f\} : \forall \vec{X}.U \rightarrow V}.$$

It follows that tree calculus, like traditional combinatory logic, supports all of the μ -recursive functions as typed programs, i.e. as typed normal forms.

Note that there are several ways of defining fixpoint functions in combinatory logic, and each choice will lead to different subtyping rules. For example, the implementation of Z could be made smaller by using

$$Z_{\text{alt}}\{f\} = \text{wait2}\{\text{self_apply}, \omega_2, f\}$$

where **self_apply** $= \lambda x.xx = S_1\{I\}I$. Then the relevant subtyping rule would act on the program type of **self_apply** instead of that of ω_2 . Similarly, we could begin with $Y_2\{f\}$ that satisfies $Y_2\{f\} x \longrightarrow f x Y_2\{f\}$ but instead we will define it by

$$Y_2\{f\} = Z\{\text{swap}\{f\}\}.$$

equal =

$$Y_2 (\text{trriage}\{\lambda e.\text{trriage}\{\text{true}, K \text{ false}, K^2 \text{ false}\}, \lambda x.\lambda e.\text{trriage}\{\text{false}, e x, K^2 \text{ false}\}, \lambda x_1.\lambda x_2.\lambda e.\text{trriage}\{\text{false}, K \text{ false}, \lambda y_1.\lambda y_2.\text{ex}_1 y_1 (\text{ex}_2 y_2) \text{ false}\}\})$$

Figure 5. Generic Equality

7 Generic Queries

Generic queries have types such as $\forall X.X \rightarrow \text{Bool}$ which produces a boolean for any argument. Typically, they are given by **trriage**. For example, the tests for being a leaf, stem or fork are given by

$$\text{isLeaf} = \text{trriage}\{\text{true}, K \text{ false}, K^2 \text{ false}\}$$

$$\text{isStem} = \text{trriage}\{\text{false}, K \text{ true}, K^2 \text{ false}\}$$

$$\text{isFork} = \text{trriage}\{\text{false}, K \text{ false}, K^2 \text{ true}\}$$

These fundamental queries will all have type $\forall X.X \rightarrow \text{Bool}$. In each case, the natural type of the **trriage** is

$$\text{F} (\text{F Bool} (\forall X.X \rightarrow \text{Bool})) (\forall X.\forall Y.X \rightarrow Y \rightarrow \text{Bool})$$

so for these examples it would be enough to make this type a subtype of $\forall Z.Z \rightarrow \text{Bool}$.

Similarly, we can define a program **size** that computes the size of a program. The required subtyping replaces **Bool** above with **Nat** to prove:

Theorem 7.1 (derive_size). **size** : $\forall X.X \rightarrow \text{Nat}$.

However, some queries take more than one argument. For example, consider the generic equality in Figure 5. It is a recursive function that performs **trriage** on each of its arguments in turn, for a total of nine cases. The program **equal** has 780 nodes: its ternary representation is given in Figure 6.

Theorem 7.2 ($\text{equality_of_programs}$). *For all programs M and N , if $M = N$ then **equal** $M N$ reduces to **tt** else it reduces to **ff**.*

Theorem 7.3 (derive_equal). **equal** : $\forall X.\forall Y.X \rightarrow Y \rightarrow \text{Bool}$.

The usual type $\forall X.X \rightarrow X \rightarrow \text{Bool}$ for **equal** can be generalised to $\forall X.\forall Y.X \rightarrow Y \rightarrow \text{Bool}$. Indeed, this generality is necessary for the type derivation. To see this, consider the equality of two forks $\Delta u_1 u_2$ and $\Delta v_1 v_2$ whose common type $X \rightarrow Z$ is given by the subtypings:

$$\text{F L Z} < X \rightarrow Z$$

$$\text{F (S (X} \rightarrow Y \rightarrow Z)) (X \rightarrow Y) < X \rightarrow Z.$$

The reduction

$$\text{equal} (\Delta u_1 u_2) (\Delta v_1 v_2) \longrightarrow \text{equal } u_1 v_1 (\text{equal } u_2 v_2) \text{ false}$$

21212120212011211010212120021200212011200212011200
 21212002120021201021200212002121200212002120102110
 102120102110102021201021101020202110102021201121101
 021212002120021201120021201120021212002120021201021
 200212002121200212002120102110102120102110102021201
 021101020202110102021201122202210202021101020202021
 101021212002120021201120021201120120211010212011211
 010212010211010202020202021101021201120112012202110
 102020211010212120021200212011200212011200212011201
 1200212011201120021201120112011200212011201120112002
 121200212002120102120021200212011200212011200212011
 2011200212011201120021201120112010212120021200212011
 2002120112002120112010212011211010212010211010202021
 1010202120112010212120021200212011200212011200212011
 2010212011211010212010211010202021101020202020202021
 101010211010

Figure 6. The Ternary Representation of **equal**

invokes **equal** $u_1 v_1$ where $u_1 : L$ and $v_1 : S$ ($X \rightarrow Y \rightarrow Z$) have incompatible types. A side-effect of this generous typing is that arguments may have different types and yet be equal. For example, **false** : **Bool** and **zero** : **Nat** are both equal to K . Hence, the type of the result of applying **equal** to an argument is not **Bool** but $Y \rightarrow \mathbf{Bool}$.

Typing of such examples can be handled by generalising from **Bool** to any type T in which X is not free, to get the subtyping rule

$$F(F T (\forall X.X \rightarrow T)) (\forall X.\forall Y.X \rightarrow Y \rightarrow T) < \forall Z.Z \rightarrow T$$

if X, Y and Z are not free in T .

Updates, however, require even more generality. For example, a generic database update should have type $\forall X.X \rightarrow X$ where the argument type X appears positively in the result type. Further, a generic update of programs may have type $\forall X.\forall Y.(X \rightarrow Y) \rightarrow (X \rightarrow Y)$ in which triage is applied to the second argument, of type X . To exploit this, swap the arguments, to get the type

$$\forall X.\forall Y.X \rightarrow (X \rightarrow Y) \rightarrow Y$$

where the argument type X is still in a positive position in $(X \rightarrow Y) \rightarrow Y$ because it has been “doubly-negated.” Thus, the general form of the subtyping rule for triage yields

$$F(F (\{L/Z\}T) (\forall X.X \rightarrow \{S X/Z\}T)) \\ (\forall X.Y.X \rightarrow Y \rightarrow \{F X Y/Z\}T) < \forall Z.Z \rightarrow T$$

where the type variable Z appears *covariantly* in the type T . Covariance is defined in the traditional manner; details are in the **Coq** implementation. Since the notation in this rule is

quite heavy, let us compress it by defining

$$\begin{aligned} \mathbf{AsLf}\{Z, T\} &= \{L/Z\}T \\ \mathbf{AsSm}\{Z, T\} &= \forall X.X \rightarrow \{S X/Z\}T \\ \mathbf{AsFk}\{Z, T\} &= \forall X.\forall Y.X \rightarrow Y \rightarrow \{F X Y/Z\}T \\ &\quad (\text{if } X, Y \text{ not free in } T). \end{aligned}$$

Then the subtyping rule becomes

$$F(F \mathbf{AsLf}\{Z, T\} \mathbf{AsSm}\{Z, T\}) \mathbf{AsFk}\{Z, T\} < \forall Z.Z \rightarrow T$$

if Z is covariant in T .

For example, the breadth-first evaluator will use a program **eager** : $(X \rightarrow Y) \rightarrow (X \rightarrow Y)$ to enforce eager evaluation of the second argument, defined by **swap**{**eager_s**} where

$$\mathbf{eager_s} = \mathbf{triage}\{\lambda f.f\Delta, \lambda x.\lambda f.f(\Delta x), \lambda w.\lambda x.\lambda f.f(\Delta wx)\}.$$

That is, **eager_s** $x f$ reduces to $f v$ if x reduces to some v which is a leaf, stem or fork. If x does not have such a value then f is never applied.

These examples implement algorithms that do not try to exploit type information but further work will be required to support generic functions such as **map** and **fold** that may act over lists or tuples, etc. Current practice may use encodings [28] or staging [43] to specialise the algorithm to each type. In a tree calculus, however, data structures can be represented as tuples that have been tagged by information about the constructor and its arguments, so that specialisation is not required.

8 Self-Interpretation

Traditionally, there have been two sorts of self-interpreters that act on encodings [22]. A *self-recogniser* of type

$$\mathbf{Exp}\{U \rightarrow V\} \rightarrow U \rightarrow V$$

performs decoding, while a *self-enactor* of type

$$\mathbf{Exp}\{U \rightarrow V\} \rightarrow \mathbf{Exp}\{U\} \rightarrow \mathbf{Exp}\{V\}$$

must actually describe how evaluation is to act on expression, by applying a strategy. Since we are not using encodings at all, self-recognisers are redundant, so the focus turns to the enactors.

The strategy we will implement is breadth-first: when evaluating a fork, both branches will be evaluated before applying any redex, much as in an eager evaluation strategy. The evaluation rules for this binary relation $\Downarrow$ on programs given in Figure 7. By contrast, a depth-first strategy would always reduce the head if possible, only evaluating branches when forced to do so.

The breadth-first evaluation strategy is internalised by the program **bf** defined in Figure 8. The fixpoint operator Z causes it to traverse its first argument. If it is a leaf or stem then the program halts. If it is a fork $\Delta f y$ then **bf** performs triage on f . If f is a leaf then halt else if it is a stem then apply **bffs** else apply **bfff**. In turn, **bffs** $x y z$ reduces to **eager** (**bf** (**bf** $x z$)) (**bf** $y z$) which forces evaluation of **bf** $y z$

$$\begin{array}{c}
\frac{}{\Delta, y \Downarrow \Delta y} \quad \frac{}{\Delta y, z \Downarrow \Delta yz} \\
\\
\frac{}{\Delta \Delta y, z \Downarrow y} \quad \frac{x, z \Downarrow u \quad y, z \Downarrow v \quad u, v \Downarrow w}{\Delta(\Delta x)y, z \Downarrow w} \\
\\
\frac{}{\Delta(\Delta wx)y, \Delta \Downarrow w} \quad \frac{x, z \Downarrow v}{\Delta(\Delta wx)y, \Delta z \Downarrow v} \\
\\
\frac{y, z_1 \Downarrow v_1 \quad v_1, z_2 \Downarrow v}{\Delta(\Delta wx)y, \Delta z_1 z_2 \Downarrow v}
\end{array}$$

Figure 7. Breadth-First Evaluation

$\mathbf{bfs} = \lambda x. \lambda y. S_1 \{S_1 \{K \text{ eager}\} (S_1 \{Ke\} (ex))\} (ey)$
 $\mathbf{bfff_fork} = \lambda y. S_1 \{Ke\} (ey)$
 $\mathbf{bfff} = \lambda w. \lambda x. \lambda y. \text{trriage}\{w, ex, \mathbf{bfff_fork} y\}$
 $\mathbf{bff} = \text{trriage}\{K, \mathbf{bfs}, \mathbf{bfff}\}$
 $\mathbf{bf} = Z\{\lambda e. \text{trriage}\{\Delta, \Delta, \mathbf{bff}\}\}$

Figure 8. The Breadth-First Evaluator

as well as $\mathbf{bf} \ x \ z$. Finally, $\mathbf{bfff} \ w \ x \ y$ reduces to $\mathbf{bf} \ (\mathbf{bf} \ y \ w) \ x$. The program $\mathbf{bf}$ has 877 nodes; its ternary representation is given in Figure 9. This concrete representation reminds us that despite the use of high-level constructions such as fixpoints and triage, all programs are actually natural binary trees. Further, it is remarkable that self-interpretation requires so little; that an evaluation strategy can be expressed by a tree with less than one thousand nodes.

Theorem 8.1 (branch_first_eval_iff_bf). *For all programs t , u and p we have $t, u \Downarrow p$ if and only if $\mathbf{bf} \ t \ u \longrightarrow p$.*

Typing of $\mathbf{bf}$ introduces significant challenges. First, consider the reduction

$$\mathbf{bf} \ (\Delta uv) \ z \longrightarrow \mathbf{bff} \ u \ v \ z.$$

If $u : U$ and $v : V$ and $z : Z$ then the typing of the left-hand side by some type T requires that $F \ U \ V < Z \rightarrow T$. So we must show that $\mathbf{bff} : U \rightarrow V \rightarrow Z \rightarrow T$ whenever $F \ U \ V < Z \rightarrow T$. However, conditional typing is awkward at best.

Second, consider the reduction

$$\mathbf{bf} \ (\Delta(\Delta u) \ v) \ z \longrightarrow \mathbf{bfs} \ u \ v \ z \longrightarrow \text{eager} \ (\mathbf{bf} \ (\mathbf{bf} \ u \ z)) \ (\mathbf{bf} \ v \ z).$$

The left-hand side has type T if $u : Z \rightarrow Y \rightarrow T$ and $v : Z \rightarrow Y$ and $z : Z$. In this case, the right-hand side will also have type T . However, this information is *not* available at the time that $\mathbf{bfs}$ is being typed. Similar remarks apply to the typing of $\mathbf{bfff}$.

212121202120112110102121200212002120112002120112002
121200212002120102120021200212120021200212010211010
2120102110102021201021101020202110102021201121101021
212002120021201120021201120021212002120021201021200
212002121200212002120102110102120102110102021201021
1201120021201120021201120102120112002120112002120112
0112021201121202221211010200212011211010212010212002
1101021201120112110102120112010212120021200212010212
0021101020211010211010212010211010212120021200212010
2120021200212010211010212120021200212010211010202110
1021201021212002120021201021101020211010212120021200
21201120021201120021201120112002120112011200212011201
1201021201120112002120112120021200212010212002110102
1201021212002120021201021101020211010212010212010212
1200212002120102121200212002120102120021200212010211
0102121200212002120102110102021101020211010211010

Figure 9. The Ternary Representation of $\mathbf{bf}$

The solution is to introduce a new type form $A \ T$ (pronounced "as fun T") which records the obligation for T to become a function type. The corresponding subtyping rules are

$$\begin{array}{c}
T < A \ T \\
A \ (U \rightarrow V) < U \rightarrow V.
\end{array}$$

Now our goal is to prove $\mathbf{bf} : \forall Z. Z \rightarrow A \ Z$. Once that is done then instantiating Z to a function type allows the tagging to be removed, so that we have

$$\mathbf{bf} : \forall X. \forall Y. (X \rightarrow Y) \rightarrow (X \rightarrow Y)$$

as desired.

Returning to our challenges, there remains a gap in the typing of $\mathbf{bfff}$ and $\mathbf{bfs}$. The intended result type for $\mathbf{bfff}$ is of the form $A \ (F \ (F \ U \ V) \ W)$ but the derived type is

$$F \ (F \ U \ (A \ V)) \ (\mathbf{bffa} \ W)$$

where

$$\mathbf{bffa} \ U = F \ (S \ (F \ L \ (\forall X. X \rightarrow A \ X))) \ (A \ U)$$

so add a subtyping rule

$$F \ (F \ U \ (A \ V)) \ (\mathbf{bffa} \ W) < A \ (F \ (F \ U \ V) \ W).$$

Similarly, the intended result type for $\mathbf{bfs}$ is of the form $A \ (F \ (S \ U) \ V)$ but the derived type is $F \ (S \ (\mathbf{bfsa} \ U)) \ (A \ V)$ where $\mathbf{bfsa} \ U$ is

$$F \ (S \ (F \ L \ (\forall X. \forall Y. (X \rightarrow Y) \rightarrow (X \rightarrow Y)))) \ (\mathbf{bffa} \ U)$$

so add a subtyping rule

$$F \ (S \ (\mathbf{bfsa} \ U)) \ (A \ V) < A \ (F \ (S \ U) \ V).$$

With these additions, $\mathbf{bf}$ can be shown to have the desired type.

9 The Richest Type System

This section will derive types for all of the examples in the paper while using all of the subtyping rules that have been considered.

The *terms* and *types* are given by

$$\begin{aligned} t, u, v &::= x \mid \Delta \mid t u \\ T, U, V &::= \mathbf{L} \mid \mathbf{S} U \mid \mathbf{F} U V \mid U \rightarrow V \mid X \mid \forall X. T \mid \mathbf{A} T. \end{aligned}$$

The term reduction rules are given in Figure 1.

Define

$$\begin{aligned} \Omega_2 &= \text{progty}(\omega_2) \\ \text{Fix } T &= (T \rightarrow T) \rightarrow T \\ \text{bfffa } U &= \mathbf{F} (\mathbf{S} (\mathbf{F} \mathbf{L} (\forall X. X \rightarrow \mathbf{A} X))) (\mathbf{A} U) \\ \text{bffsa } U &= \mathbf{F} (\mathbf{S} (\mathbf{F} \mathbf{L} (\forall X. \forall Y. (X \rightarrow Y) \rightarrow (X \rightarrow Y)))) \\ &\quad (\text{bfffa } U). \end{aligned}$$

The subtyping rules are generated from the subtyping axioms in Figure 10 and closed under the type formation operations (argument types are contravariant in function types) and used to generate a reflexive, transitive order. Most of the axioms have been discussed earlier. The remaining four allow quantifiers to distribute over stems, forks, function types and **A** types.

The type derivation rules are given in Figure 4. When the context Γ is empty, we may write $t : T$ for $\Gamma \vdash t : T$. Although subtyping appears in the derivations for term variables and nodes only, any type derivation can be subtyped, as follows.

Theorem 9.1 (derive_subtype). *For each type context Γ and term t and types T_1 and T_2 , if $\Gamma \vdash t : T_1$ and $T_1 < T_2$ then $\Gamma \vdash t : T_2$.*

All of the theorems about tree calculus in earlier sections can be proved in this system, as shown in the attached **Coq** proofs.

10 Reduction Preserves Typing

It remains to prove that reduction preserves typing by examining the reduction rules. For example, consider the reduction $Kuv \rightarrow u$, where $u : U$ and $v : V$. If $\mathbf{F} \mathbf{L} U < V \rightarrow T$ then $Kuv : T$ so we must show that $u : T$ i.e. that $U < T$. The next five theorems handle all of the different cases. Then the final theorem shows that reduction preserves typing.

Theorem 10.1 (subtype_from_fork_of_leaf_to_fun). *$\mathbf{F} \mathbf{L} V < Z \rightarrow T$ implies $V < T$ for all types V, Z and T .*

Theorem 10.2 (subtype_from_fork_of_stem_to_funty). *For all types U, V, Z and T if $\mathbf{F} (\mathbf{S} U) V < Z \rightarrow T$ then there is a type Y such that $U < Z \rightarrow Y \rightarrow T$ and $V < Z \rightarrow Y$.*

Theorem 10.3 (subtype_from_fork_of_fork_of_leaf). *For all types W, U, V and T , if $\mathbf{F} (\mathbf{F} W U) V < \mathbf{L} \rightarrow T$ then $W < T$.*

$$\begin{aligned} \mathbf{L} &< U \rightarrow \mathbf{S} U \\ \mathbf{S} U &< V \rightarrow \mathbf{F} U V \\ \mathbf{F} \mathbf{L} U &< V \rightarrow U \\ \mathbf{F} (\mathbf{S} (U \rightarrow V \rightarrow T)) (U \rightarrow V) &< U \rightarrow T \\ \mathbf{F} (\mathbf{F} U V) W &< \mathbf{L} \rightarrow U \\ \mathbf{F} (\mathbf{F} U (V_1 \rightarrow V_2)) W &< \mathbf{S} V_1 \rightarrow V_2 \\ \mathbf{F} (\mathbf{F} U V) (W_1 \rightarrow W_2 \rightarrow W_3) &< \mathbf{F} W_1 W_2 \rightarrow W_3 \\ \forall X. \mathbf{S} U &< \mathbf{S} (\forall X. U) \\ \forall X. \mathbf{F} U V &< \mathbf{F} (\forall X. U) (\forall X. V) \\ \forall X. \mathbf{A} U &< \mathbf{A} (\forall X. U) \\ \forall X. U \rightarrow V &< (\forall X. U) \rightarrow (\forall X. V) \\ \forall X. T &< \{U/X\}T \\ T &< \forall X. T \quad (X \notin T) \\ \Omega_2 &< \Omega_2 \rightarrow \text{Fix } (\forall \vec{X}. U \rightarrow V) \\ \mathbf{F} (\mathbf{F} \text{AsLf}\{Z, T\} \text{AsSm}\{Z, T\}) \\ \text{AsFk}\{Z, T\} &< U \rightarrow \{U/Z\}T \\ &\quad (\text{if } Z \text{ is covariant in } T) \\ T &< \mathbf{A} T \\ \mathbf{A} (U \rightarrow V) &< U \rightarrow V \\ \mathbf{F} (\mathbf{S} (\text{bffsa } U)) (\mathbf{A} V) &< \mathbf{A} (\mathbf{F} (\mathbf{S} U) V) \\ \mathbf{F} (\mathbf{F} U (\mathbf{A} V)) (\text{bfffa } (\mathbf{A} W)) &< \mathbf{A} (\mathbf{F} (\mathbf{F} U V) W). \end{aligned}$$

Figure 10. Subtyping Axioms

Theorem 10.4 (subtype_from_fork_of_fork_of_stem). *For all types W, U, V, Z and T , if $\mathbf{F} (\mathbf{F} W U) V < \mathbf{S} Z \rightarrow T$ then $U < Z \rightarrow T$.*

Theorem 10.5 (subtype_from_fork_of_fork_of_fork). *For all types $W, U, V, Z_1 Z_2$ and T , if $\mathbf{F} (\mathbf{F} W U) V < \mathbf{F} Z_1 Z_2 \rightarrow T$ then $V < Z_1 \rightarrow Z_2 \rightarrow T$.*

Theorem 10.6 (reduction_preserves_typing). *For all type context Γ and terms t_1 and t_2 and type T , if $\Gamma \vdash t_1 : T$ and $t_1 \rightarrow t_2$ then $\Gamma \vdash t_2 : T$.*

It is important to note that none of the proofs of the theorems above is straightforward. Rather, they follow from a complete characterisation of the subtyping relation which must address several sources of complexity. First, quantifiers can be introduced and eliminated at any point: these can be combined into type substitutions which are passed as parameters. Second, **A** can be introduced and eliminated at any point. Third, the special subtyping rules for recursion, generic queries and **bf** must be shown to respect typing. For example, if the given subtyping in Theorem 10.2 is $\Omega_2 < \Omega_2 \rightarrow \text{Fix } T$ then the proof of the theorem mirrors the proof that $Z\{-\}$ matches its specification. Finally, the main theorem relies on a relatively routine characterisation

of type derivation. The details of the development are too verbose for inclusion in the main body of the paper but can be found in the related **Coq** code.

11 Conclusions and Future Work

Unlike traditional combinatory logic, tree calculus supports program analysis as well as numerical analysis, without invoking any of the encodings or meta-theory that is usually required. This is because the programs are exactly the (closed) normal forms, given as natural binary trees, whose internal structure can be explored using generic queries, such as equality, or manipulated by, say, a self-interpreter. Rather, the inherent tension between the views of programs as data structures or as functions re-emerges in the type theory. Although two derivation axioms are enough to type all programs, they are parametrised by a subtyping relation that is used to convert tree types to functions types. More precisely, any cleverness that is used to show correctness of a program must be captured by the subtyping rules, as was demonstrated for recursive functions and self-interpreters.

Much remains to be done. It will be interesting to see how systematically partial evaluation [21] and other program analyses, e.g. [11, 42] can be developed and typed without encodings. Theoretically, this account of typed tree calculus is still using meta-theory to represent the types and the type-level computations. However, it should be possible to represent types, typed terms and type-level computations within tree calculus by using tags. Among other things, they can be used to demarcate the boundaries between shapes and the data they hold [23, 31] to support generic mapping and folding, etc. *Dependent types* and *dynamic types* [14] should be equally supported in this setting.

Indeed, perhaps we can even support self-inference, as given by self-application of a program **infer** such that

$$\text{infer } p \ U = \text{Some } V$$

implies $p : U \rightarrow V$.

In another direction, program analysis becomes complexity theory, which estimates the execution time of programs, relative to the size of their inputs. These sizes are easily computed in the Turing model, but are unstable in lambda calculus, since the input to a higher-order function may be a program that does not have a normal form. In tree calculus, however, we have defined the program **size** and so could develop programs to compute complexity of higher-order programs.

12 Data-Availability Statement

The proofs of all theorems have been verified in Coq [20].

References

- [1] H.P. Barendregt. 1981. *The Lambda Calculus: Its Syntax and Semantics*. North Holland.
- [2] Henk Barendregt. 1991. Self-interpretations in lambda calculus. *J. Functional Programming* 1, 2 (1991), 229–233.
- [3] Bruno Barras, Samuel Boutin, Cristina Cornes, Judicaël Courant, Jean-Christophe Filliâtre, Eduardo Giménez, Hugo Herbelin, Gérard Huet, César Muñoz, Chetan Murthy, Catherine Parent, Christine Paulin-Mohring, Amokrane Saïbi, and Benjamin Werner. 1997. *The Coq Proof Assistant Reference Manual : Version 6.1*. Research Report RT-0203. INRIA. 214 pages. <https://hal.inria.fr/inria-00069968> Projet COQ.
- [4] Alessandro Berarducci and Corrado Böhm. 1993. *A self-interpreter of lambda calculus having a normal form*. Springer Berlin Heidelberg, Berlin, Heidelberg, 85–99.
- [5] Matt Brown and Jens Palsberg. 2015. Self-Representation in Girard’s System U. In *Proceedings of the 42Nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Mumbai, India) (POPL ’15). ACM, New York, NY, USA, 471–484.
- [6] Matt Brown and Jens Palsberg. 2016. Breaking Through the Normalization Barrier: A Self-interpreter for F-omega. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (St. Petersburg, FL, USA) (POPL ’16). ACM, New York, NY, USA, 5–17.
- [7] Luca Cardelli and Peter Wegner. 1985. On understanding types, data abstraction, and polymorphism. 17, 4 (Dec. 1985), 471–523. <https://doi.org/10.1145/6041.6042>
- [8] Jacques Carette, Oleg Kiselyov, and Chung-Chieh Shan. 2009. Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages. *Journal of Functional Programming* 19 (9 2009), 509–543. Issue 05. http://journals.cambridge.org/article_S0956796809007205
- [9] A. Church. 1941. *The Calculi of Lambda-Conversion*. Princeton University Press, Princeton, New Jersey.
- [10] H.B. Curry and R. Feys. 1958. *Combinatory Logic*. Combinatory Logic, Vol. 1. North-Holland Publishing Company.
- [11] Paul Downen, Philip Johnson-Freyd, and Zena M Ariola. 2020. Abstracting models of strong normalization for classical calculi. *Journal of Logical and Algebraic Methods in Programming* 111 (2020), 100512.
- [12] Solomon Feferman. 1975. A language and axioms for explicit mathematics. In *Algebra and Logic*, John Newsome Crossley (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 87–139.
- [13] J-Y. Girard, Y. Lafont, and P. Taylor. 1989. *Proofs and Types*. Cambridge University Press.
- [14] Fritz Henglein. 1992. Dynamic typing. In *ESOP ’92*, Bernd Krieg-Brückner (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 233–253.
- [15] R. Hindley and J.P. Seldin. 1986. *Introduction to Combinators and Lambda-calculus*. Cambridge University Press.
- [16] Barry Jay. 2009. *Pattern Calculus: Computing with Functions and Structures*. Springer.
- [17] Barry Jay. 2018. Recursive Programs in Normal Form (Short Paper). In *Proceedings of the ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation (PEPM ’18)*. ACM, 67–73.
- [18] Barry Jay. 2019. Intensional computation with higher-order functions. *Theoretical Computer Science* 768 (2019), 76–90.
- [19] Barry Jay. 2021. *Reflective Programs in Tree Calculus*. <https://github.com/barry-jay-personal/tree-calculus> including an appendix with Jose Vergara.
- [20] Barry Jay. 2024. TypedProgramAnalysis: repository of proofs in Coq. <https://doi.org/10.1145/3580441>
- [21] Barry Jay and Thomas Given-Wilson. 2011. A Combinatory Account of Internal Structure. *Journal of Symbolic Logic* 76, 3 (2011), 807–826.
- [22] Barry Jay and Jens Palsberg. 2011. Typed Self-Interpretation by Pattern Matching. In *Proceedings of the 2011 ACM Sigplan International Conference on Functional Programming*. 247–58.
- [23] C.B. Jay. 1995. A semantics for shape. *Science of Computer Programming* 25 (1995), 251–283.

- [24] Ende Jin, Nada Amin, and Yizhou Zhang. 2023. Extensible metatheory mechanization via family polymorphism. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 1608–1632.
- [25] N.D. Jones, C.K. Gomard, and P. Sestoft. 1993. *Partial Evaluation and Automatic Program Generation*. Prentice Hall International.
- [26] S. C. Kleene. 1936. λ -definability and recursiveness. *Duke Math. J.* 2, 2 (06 1936), 340–353.
- [27] Stephen C. Kleene. 1936. λ -Definability and Recursiveness. *Duke Mathematical Journal* 2 (1936), 340–353.
- [28] Hsiang-Shang Ko, Liang-Ting Chen, and Tzu-Chi Lin. 2022. Datatype-generic programming meets elaborator reflection. *Proceedings of the ACM on Programming Languages* 6, ICFP (2022), 225–253.
- [29] András Kovács. 2022. Staged compilation with two-level type theory. *Proc. ACM Program. Lang.* 6, ICFP, Article 110 (Aug. 2022), 30 pages. <https://doi.org/10.1145/3547641>
- [30] Konstantin Läuffer and Martin Odersky. 1993. Self-interpretation and reflection in a statically typed language. In *OOPSLA/ECOOP workshop on object-oriented reflection and metalevel architectures*. Citeseer.
- [31] Kiminori Matsuzaki, Zhenjiang Hu, and Masato Takeichi. 2006. Parallel skeletons for manipulating general trees. *Parallel Comput.* 32, 7-8 (2006), 590–603.
- [32] Torben Æ. Mogensen. 1992. Efficient Self-Interpretations in Lambda Calculus. *Journal of Functional Programming* 2, 3 (1992), 345–363. See also DIKU Report D-128, Sep 2, 1994.
- [33] Torben Æ. Mogensen. 2000. Linear-Time Self-Interpretation of the Pure Lambda Calculus. *Higher-Order and Symbolic Computation* 13, 3 (2000), 217–237.
- [34] Flemming Nielson and Hanne Riis Nielson. 1992. *Two-level functional languages*. Cambridge University Press, USA.
- [35] Tillmann Rendel, Klaus Ostermann, and Christian Hofer. 2009. Typed self-representation. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Dublin, Ireland) (PLDI '09). Association for Computing Machinery, New York, NY, USA, 293–303. <https://doi.org/10.1145/1542476.1542509>
- [36] John C. Reynolds. 1972. Definitional Interpreters for Higher-Order Programming Languages. In *Proceedings of 25th ACM National Conference*. ACM Press, 717–740. The paper later appeared in *Higher-Order and Symbolic Computation*.
- [37] Moses Schönfinkel. 1924. Über die Bausteine der mathematischen Logik. *Math. Ann.* 92 (1924), 305–316.
- [38] Fangmin Song, Yongsun Xu, and Yuechen Qian. 2000. The self-reduction in lambda calculus. *Theoretical Computer Science* 235, 1 (2000), 171 – 181.
- [39] Wenhao Tang, Daniel Hillerström, James McKinna, Michel Steuwer, Ornela Dardha, Rongxiao Fu, and Sam Lindley. 2023. Structural subtyping as parametric polymorphism. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2 (2023), 1093–1121. <https://doi.org/10.1145/3622836>
- [40] Terese. 2003. *Term Rewriting Systems*. Tracts in Theoretical Computer Science, Vol. 53. Cambridge University Press.
- [41] A. M. Turing. 1936. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* 2 42 (1936), 230–65.
- [42] Guannan Wei, Oliver Bračevac, Shangyin Tan, and Tiark Rumpf. 2020. Compiling symbolic execution with staging and algebraic effects. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–33. <https://doi.org/10.1145/3428232>
- [43] Jeremy Yallop. 2017. Staged generic programming. *Proceedings of the ACM on Programming Languages* 1, ICFP (2017), 1–29. <https://doi.org/10.1145/3110273>
- [44] Furqan Zahoor, Ramzi A. Jaber, Usman Bature Isyaku, Trapti Sharma, Faisal Bashir, Haider Abbas, Ali S. Alzahrani, Shagun Gupta, and Mehwish Hanif. 2024. Design implementations of ternary logic systems: A critical review. *Results in Engineering* 23 (2024), 102761. <https://doi.org/10.1016/j.rineng.2024.102761>

Received 2024-10-14; accepted 2024-11-18